

Árvores - Introdução (1)

- As árvores são estruturas de dados usadas em diversas aplicações:
 - Armazéns de grandes volumes de dados
 - Verificação de frases fazendo parte de linguagens (ex: programas, expressões aritméticas,...)
 - Modelação de sistemas organizacionais (ex: famílias, directórios de um computador, hierarquia de comando de uma empresa,...)

Árvores - Introdução (2)

- Def: Uma **árvore** é um caso especial de um grafo, i.e., um par (V,E) de dois conjuntos não vazios V -nós (``vertex'') e  $E \subseteq V^2$ -arestas (``edges'') que satisfazem as condições:
 - Entre dois nós existe um único caminho (um **caminho** (``path'') é uma sequência de arestas entre dois nós),
 - Um único nó, denominado **raíz** (``root''), só existe como primeiro elemento nos pares de E : os restantes nós são um segundo elemento dos pares de E (podendo também ser primeiro elemento de alguns pares).

Árvores - Introdução (3)

- Def: se $(v_1, v_2) \in E$, v_1 é nó **ascendente** (``parent'') e  $v_2$  é nó **filho** (``child'').
 - A raíz é o único nó sem ascendente.
 - Nós sem filhos são designados por **folhas** (``leaves'').
 - Nós com filhos são designados **não-terminais**. Nós não-terminais, distintos da raíz, são designados **intermédios**.
- Def: A árvore é de tipo K , sse todos os nós intermédios tiverem K filhos. Quando $K=2$, a árvore diz-se **binária**.

Árvores - Introdução (4)

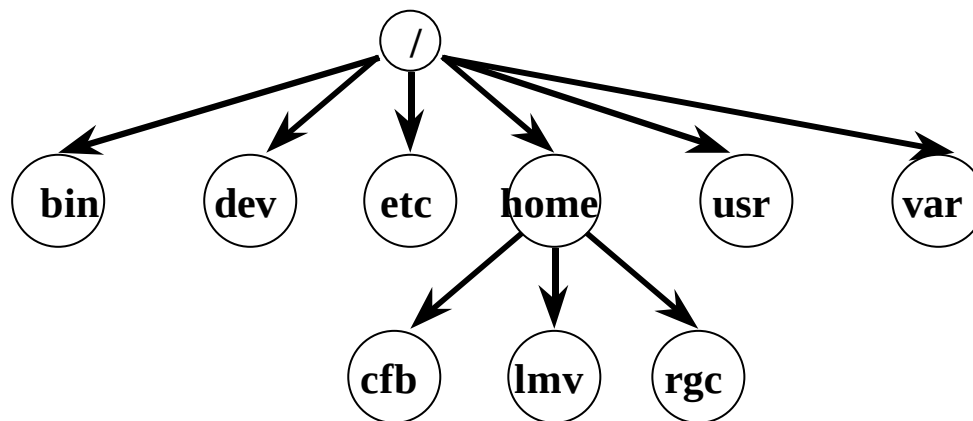
- Def: O **nível** (``level'') de um nó é o número de arestas percorridas entre a raíz e o nó. A raíz possui o nível 0.
- A **altura** (``heigh'') de uma árvore é o máximo número de nós encontrados nos caminhos até às folhas. Uma árvore só com a raíz possui altura de 1.

Árvores - Introdução (5)

Representação gráfica das árvores adopta a convenção:

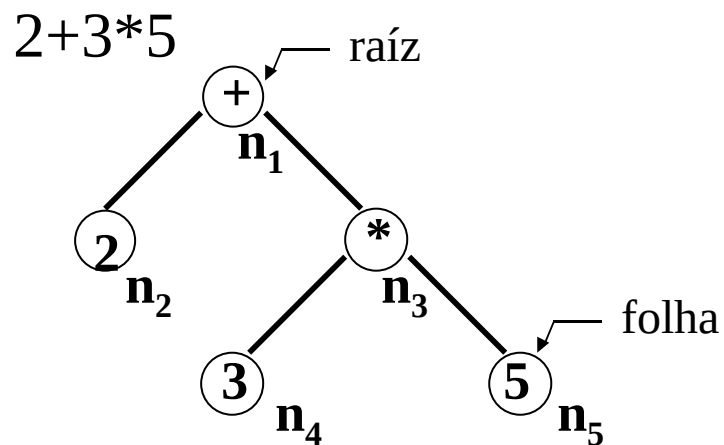
- raíz no topo,
- nós representados por círculos,
- arestas representadas por linhas (ou setas sentido raíz para folha).

Exemplo A1: directórios do sistema operativo Linux



Árvores - Introdução (6)

Exemplo A2: expressões aritméticas



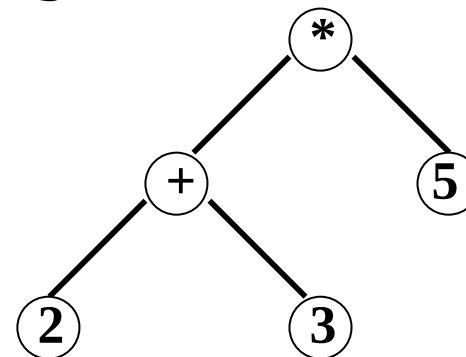
$$V = \{n_1, n_2, n_3, n_4, n_5\}$$

$$E = \{(n_1, n_2), (n_1, n_3), (n_3, n_4), (n_3, n_5)\}$$

A árvore é avaliada de forma ascendente (``bottom-up``):

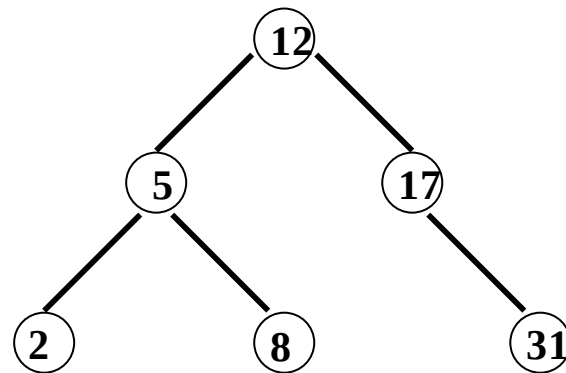
$$2+3*5 = 2+15 = 17$$

$(2+3)*5$



Árvores - Introdução (7)

Exemplo A3: árvore ordenada de inteiros



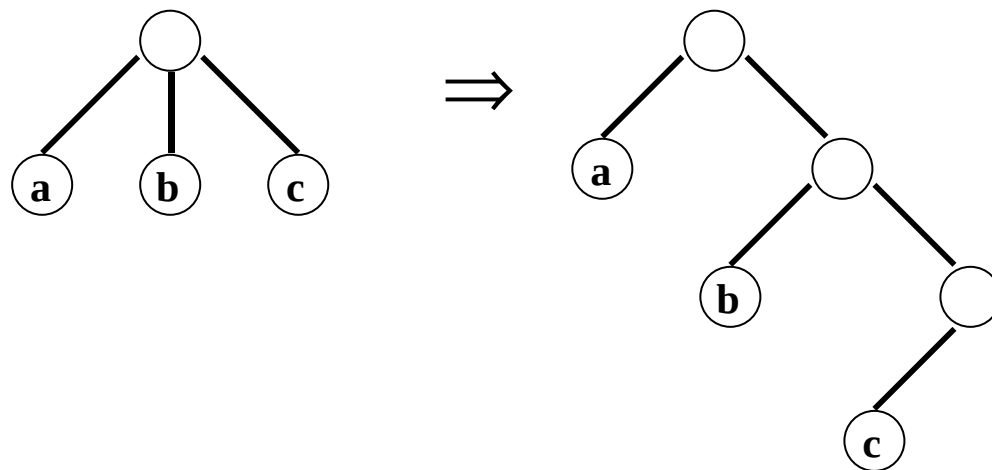
- A subárvore direita (esquerda) armazena números maiores (menores) que o nó em causa.
- A árvore também serve para armazenar identificadores (ordem lexicográfica: $cf > cb$ e $cf > az$)

Árvores - Introdução (8)

Uma árvore de ordem K pode ser transformada numa árvore binária, com

- referências esquerdas contêm sucessivamente as subárvores da árvore K
- referências direitas contém referência para a parte restantes da árvore K

Exemplo A4: transformação de árvore ternária



Árvores - Representação em C

Seja a definição em C de uma árvore binária, contendo cada nó um dado de tipo `data`.

```
typedef struct _s1 {  
    data item; /* dados */  
    struct _s1 *left, *right; /* referência filhos */  
} node;  
node *root=NULL; /* raiz (árvore inicial é vazia) */
```

Árvores - Propriedades (1)

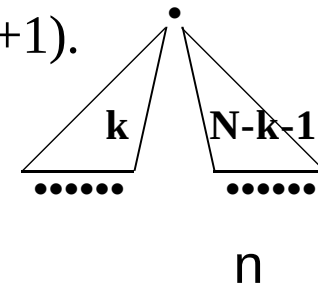
Teorema A1: Uma árvore binária com N nós não-terminais possui $N+1$ folhas.

Estratégia de prova: indução, com base na construção da árvore a partir de duas subárvores.

- Se $N=0$, a árvore possui um único nó (que é raiz e folha em simultâneo).
- Seja $N>0$ o número de nós não-terminais: a subárvore esquerda tem k nós, a subárvore direita tem $N-k-1$ nós ($0 < k < N+1$).

Por hipótese, a subárvore esquerda tem $k+1$ folhas e a subárvore direita tem $N-k$ folhas.

Somando, a árvore tem $(k+1)+(N-k)=N+1$ folhas.



Árvores - Propriedades (2)

Teorema A2: Uma árvore binária com N nós não-terminais possui $2*N$ arestas ($N-1$ para os nós não-terminais e $N+1$ para as folhas).

Estratégia de prova: contar o número de arestas para cada nó.

- Exceptuando a raiz, cada nó possui um único ascendente, pelo que só há uma aresta entre um nó e o seu ascendente.
- Pelo teorema A1 e observação anterior, há $N+1$ arestas para as folhas. Pelas duas observações anteriores, há $N-1$ arestas para os nós intermédios. No total, a árvore com N nós não-terminais possui $(N+1)+(N-1)=2*N$ arestas.

n

Árvores - Propriedades (3)

Teorema A3: Numa árvore binária com N nós, o nível das folhas h_l varia entre $\lfloor \log_2 N \rfloor$ e $N-1$.

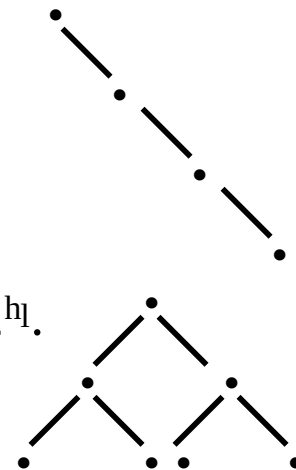
Estratégia de prova: identificar níveis máximo e mínimo.

- O nível máximo é o da árvore degenerada numa lista.

Neste caso, $h_l = N-1$ arestas.

- O nível mínimo é o da árvore balanceada, cada nível i com 2^i nós. Por A1 há $N+1$ folhas, logo $2^{h_l-1} < N+1 \leq 2^{h_l}$.

Assim, h_l é o maior inteiro igual ou inferior a $\log_2 N$
(ex: $\lfloor \log_2 7 \rfloor = 2$, $\lfloor \log_2 8 \rfloor = 3$)



Árvores - Inserção (1)

- Sendo as árvores recursivas, as funções de manipulação podem ser recursivas: a instrução recursiva é aplicada nos nós intermédios e a instrução terminal nas folhas.
- As árvores são, usualmente, construídas de forma descendente (``top-down``).

Nos algoritmos de inserção consideramos o item um inteiro (`typedef int data;`) e a árvore ordenada. Se a informação a armazenar for uma estrutura, um dos campos designado **chave** (``key``) tem de ser tipo ordenado. Ex: no bilhete de identidade, a chave é o número.

Árvores - Inserção (2)

Inserção directa

Dividida em duas partes, conforme situação da árvore:

- vazia: insere directamente novo dado
- não vazia: procura primeiro nó onde inserir novo dado

```
node *insert(int i, node *tree) {  
  node *upper=tree, *child=tree, *leaf;  
  
  if (tree==NULL) { /* árvore vazia */  
    tree=(node *) malloc(sizeof(node *));  
    tree->item=i; /* insere dado */  
    tree->left=tree->right=NULL; /* subárvores vazias */  
    return tree;}
```

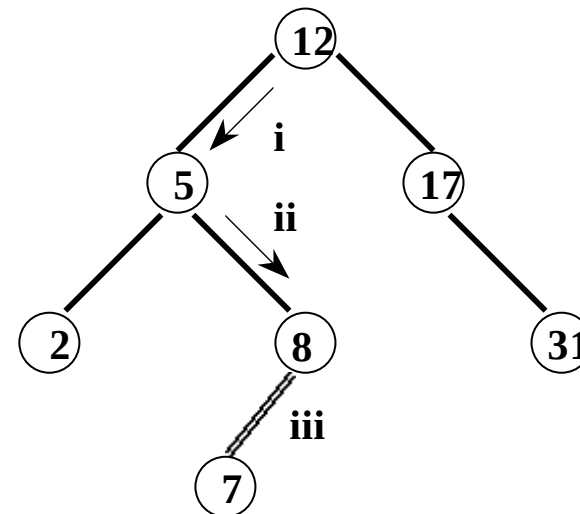
Árvores - Inserção (3)

```
while (child!=NULL){/* procura nó onde inserir dado */  
    upper=child;  
    if (child->item==i) return tree; /* já existe! */  
    child=child->item>i?child->left:child->right;}  
  
    /* criada e inicializada nova folha */  
    leaf=(node *) malloc(sizeof(node *));  
    leaf->item=i; leaf->left=leaf->right=NULL;  
  
    /* upper passa a nó intermédio e aponta para nova folha */  
    if (upper->item>i) upper->left=leaf;  
    else upper->right=leaf;  
  
    return tree; }  
  
/* rotina insert retorna nova árvore, porque a inicial foi alterada */
```

Árvores - Inserção (4)

Exemplo A5: inserção de 7 na árvore do exemplo A3

- i) $7 < 12$, insere na subárvore esquerda.
- ii) $7 > 5$, insere na subárvore direita.
- iii) 8 é folha:
 - cria nova folha.
 - $7 < 8$, a nova folha é subárvore esquerda.



O algoritmo gera uma lista, se os dados inseridos forem sucessivamente crescentes (ou decrescentes). A complexidade é determinada pela pesquisa do local de inserção, ou seja, $O(\log N)$.

Árvores - Inserção (5)

Inserção balanceada

- Def: Uma árvore diz-se **balanceada**, ou de tipo AVL¹, sse em todos os nós a diferença entre as alturas das subárvores for igual ou inferior a 1.

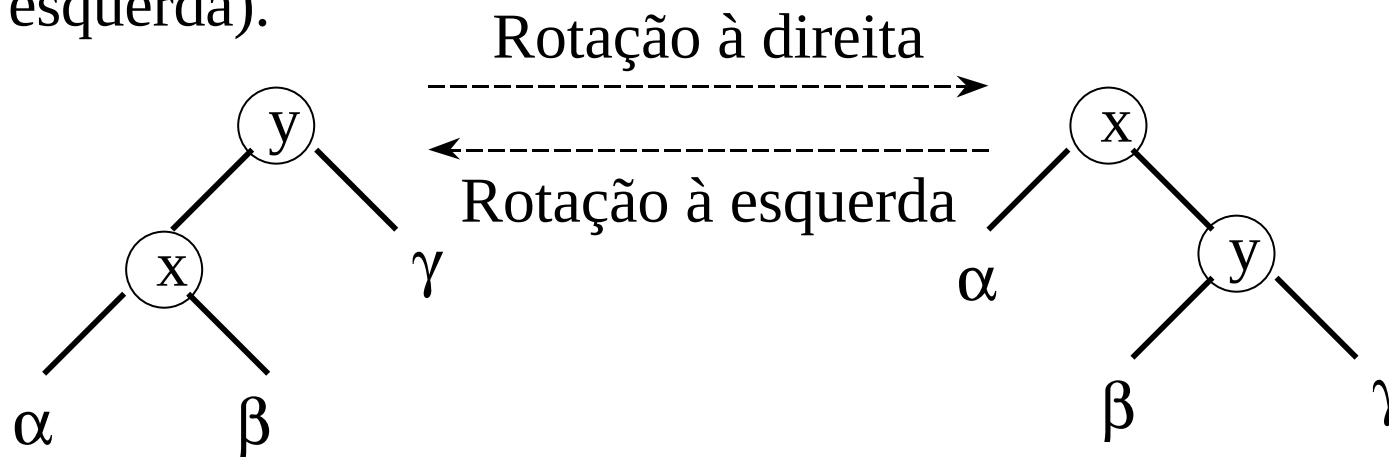
Exemplo A6: árvore A3 é balanceada, A4 já não é (subárvore esquerda tem altura 2 e direita tem altura 4).

Para manter a árvore balanceada, respeitando a ordem, torna-se necessário rodar a configuração de nós.

¹Adel'son-Vel'skii e Landis

Árvores - Inserção (6)

Operações de rotação classificadas pelo sentido (à direita e à esquerda).



Quando a diferença de alturas for igual a 2, a operação de rotação diminui o nível da folha mais afastada da raiz, mantendo a ordenação da árvore.

Árvores - Inserção (7)

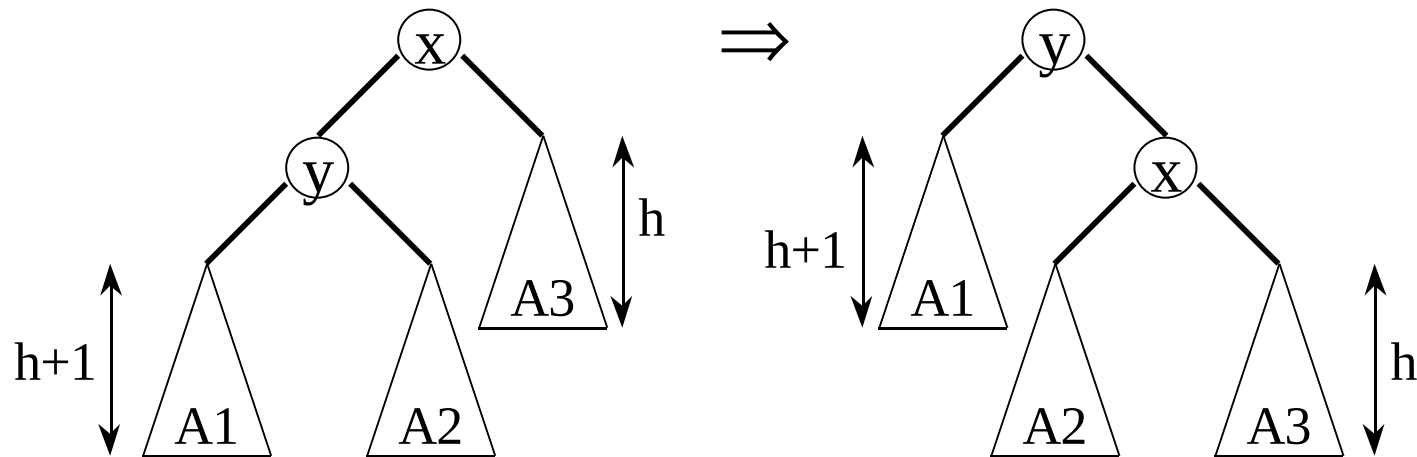
A função de rotação apenas actualiza ponteiros (β é a única subárvore a mudar de ascendente). Assim, a rotação tem complexidade $O(1)$.

```
node *rotate_right(node * tree) {  
    node *x, *y, *beta;  
    /* inalterada se árvore vazia, ou subárvore esquerda vazia */  
    if (tree==NULL) return tree;  
        else if (tree->left==NULL) return tree;  
    /* salva ponteiros */  
    y=tree; x=tree->left; beta=x->right;  
    /* actualiza ligações */  
    x->right=y; y->left=beta;  
    return x;}
```

Árvores - Inserção (8)

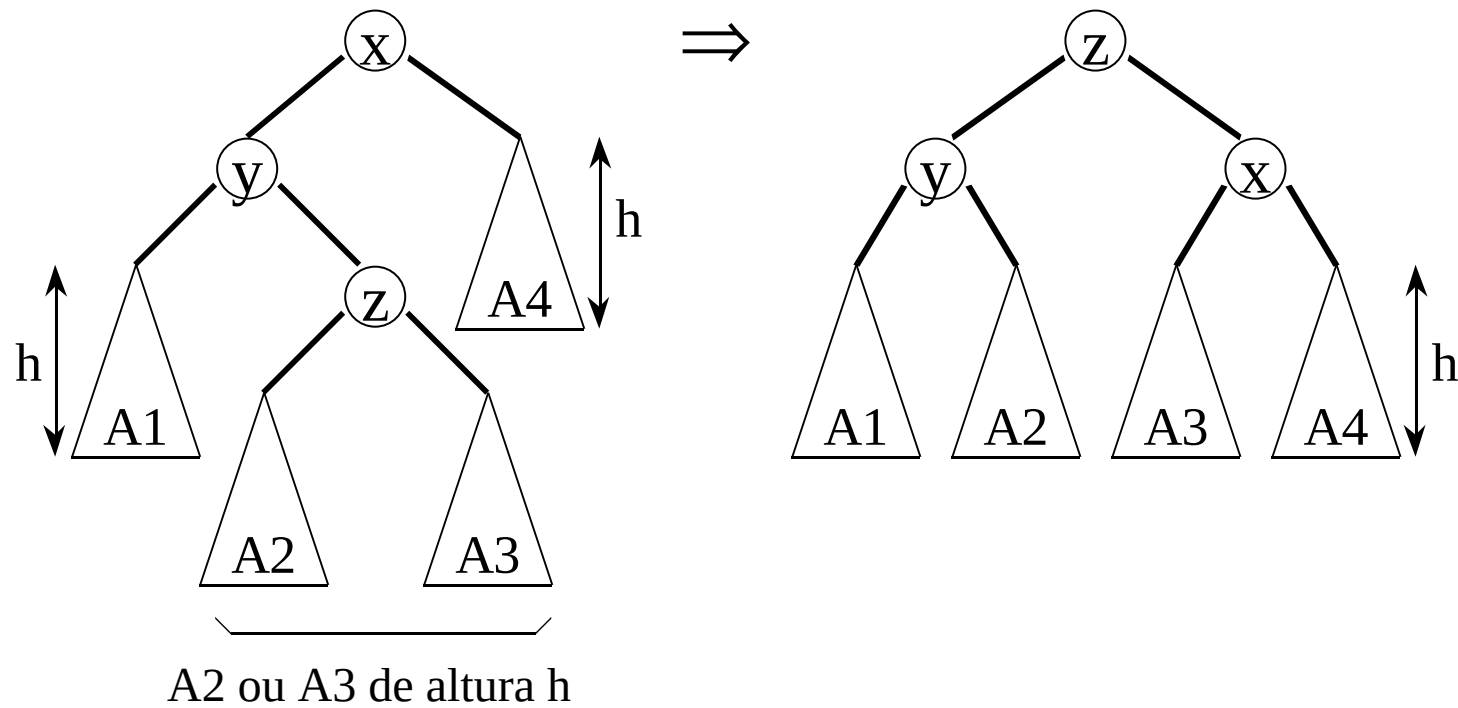
Há dois pares de casos em que se torna necessário rodar subárvores.

Caso A



Árvores - Inserção (9)

Caso B



Árvores - Inserção (10)

```
int height(node *tree) {  
    int hl,hr;  
  
    if (tree==NULL) return 0;  
    if (tree->left==NULL && tree->right==NULL)  
        return 1;  
    hl=height(tree->left); hr=height(tree->right);  
    return hl>hr ? 1+hl: 1+hr; }
```

Árvores - Inserção (11)

```
void insert(int i, node **tree) {  
    /* tree é ponteiro para árvore: a alteração é feita directamente na  
       árvore que é dada como parâmetro */  
    int h1,h2,h3;  
  
    if (*tree==NULL) { /* arvore vazia */  
        *tree=(node*) malloc(sizeof(node));  
        (*tree)->item=i;  
        (*tree)->left=(*tree)->right=NULL;  
        return; }
```

Árvores - Inserção (12)

```
if (i==(*tree)->item) return; /* já instalado */
if (i<(*tree)->item){ /* insere na subárvore esquerda */
    insert(i,&((*tree)->left));
    h1=height((*tree)->left->left);
    h2=height((*tree)->left->right);
    h3=height((*tree)->right);
    if (h1>h3) /* Caso A, rotação à direita */
        (*tree)=rotate_right(*tree);
    if (h2>h3) { /* Caso B, rotação à direita */
        (*tree)->left=rotate_left((*tree)->left);
        (*tree)=rotate_right(*tree); } }
```

Árvores - Inserção (13)

```
else { /* insere na subárvore direita */
    insert(i, &((*tree)->right));
    h1=height((*tree)->right->right);
    h2=height((*tree)->right->left);
    h3=height((*tree)->left);
    if (h1>h3) /* Caso A, rotação à esquerda */
        (*tree)=rotate_left(*tree);
    if (h2>h3) { /* Caso B, rotação à esquerda */
        (*tree)->right=rotate_right((*tree)->right);
        (*tree)=rotate_left(*tree); } } }
```

Árvores - Inserção (14)

Notas:

- O algoritmo AVL não garante distribuição total pelos níveis intermédios (por exemplo, na série $S_0 + (-1)^n n$, a subárvore esquerda só deriva à esquerda e a subárvore direita só deriva à direita).
- O algoritmo de inserção listado é pesado devido à determinação das alturas. Uma versão mais eficiente (embora mais complexa e ocupando mais memória), guarda em cada nó o balanceamento das subárvores (maior à esquerda, iguais ou maior à direita).

Árvores - Percurso (1)

Existem diversas estratégias de percurso/varrimento

(``transverse``) de árvores:

- De acordo com o operador, existente em todos os nós que não sejam folhas: o percurso é também designado por ``dump``.
 - Pré-fixado: antes do percurso das subárvores.
 - In-fixado: percorrer primeiro subárvore esquerda, imprimir operador e percorrer depois a subárvore direita.
 - Pós-fixado : depois do percurso das subárvores
- Profundidade (``depth-first``)
- Largura (``breadth-first``)

Árvores - Percurso (2)

De acordo com o operador

As subárvores percorridas de forma ordenada e o operador listado numa forma determinada.

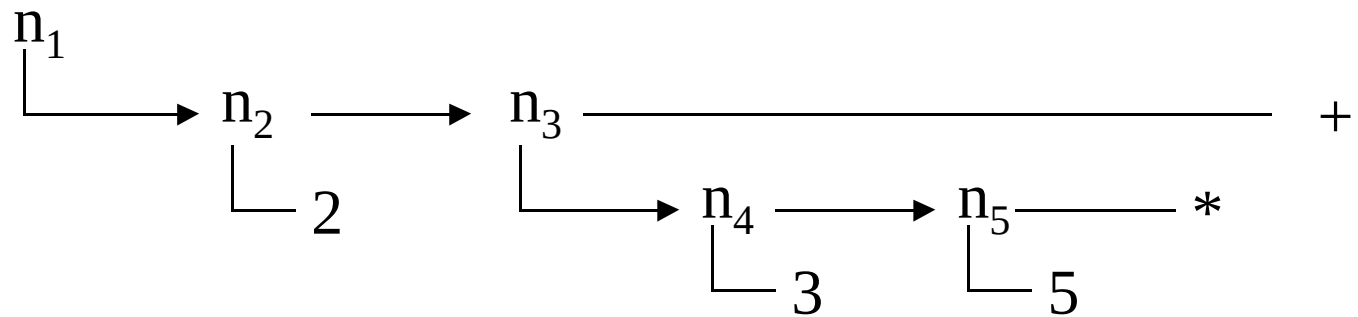
Exemplo A7: percurso pós-fixado da árvore aritmética A2

```
typedef struct {  
    enum {nod, leaf} node_type;  
    union {  
        enum {plus, time} oper;  
        int value; } info;  
} data;
```

Árvores - Percurso (3)

```
void postfix (node *tree){  
    if (tree==NULL) return;  
    switch(tree->item.node_type) {  
        case leaf:printf(" %d",tree->item.info.value);  
                    break;  
        case nod: /* imprime subárvores esq e dir */  
                    postfix(tree->left);postfix(tree->right);  
                    switch(tree->item.info.oper) { /* imprime oper */  
                        case plus: printf(" +"); break;  
                        case times: printf(" *"); break; }}}}
```

Árvores - Percurso (4)



o resultado final é: **2 3 5 * +**

Árvores - Percurso (5)

Profundidade

Idêntico ao percurso pré-fixado

```
void depth_first (node *tree){  
    if (tree==NULL) return;  
    printf(" %d", tree->item);  
    depth_first(tree->left);  
    depth_first(tree->right); }
```

Exemplo A8: O percurso em profundidade da árvore A3 executa os passos: (a) elemento da raíz - 12 (b) visita em profundidade subárvore esquerda - 5 2 8 (c) visita em profundidade subárvore direita - 17 31. O resultado é **12 5 2 8 17 31**

Árvores - Percurso (6)

Largura

O percurso é feito por nível. As subárvores são colocadas no fim de uma lista. A lista contém ponteiros para as subárvores.

```
typedef struct _s2 {  
    node *this;  
    struct _s2 *next; } link;  
link **New(); /* cria lista vazia */  
void InsertLast(link **, node *); /* insere no fim da lista */  
link *GetFirst(link **); /* obtém primeiro elemento da lista */
```

Árvores - Percurso (7)

```
void breadht_first (link **list){  
    if (list==NULL) return;  
    else {  
        link * aux = GetFirst(list);  
        printf(" %d",aux->this->item);  
        InsertLast(list,tree->this->left);  
        InsertLast(list,tree->this->right);  
        breadht_first(list); } }
```

Rotina deve ser chamada com a raíz inserida na lista.

```
breadht_first(InsertLast(New(),root));
```

Árvores - Percurso (7)

Exemplo A9: percurso em largura da árvore A3

Lista	Acção	Saída				
[12]	Recolhe 1º elemento					
	Imprime valor	12				
[5, 17]	Insere subárvores esq e dir					
[17]	Recolhe 1º elemento					
	Imprime valor	5				
[17,2,8]	Insere subárvores esq e dir					
[2,8]	Recolhe 1º elemento					
	Imprime valor	17				
[2,8,31]	Insere subárvores esq e dir					
[8,31]	Recolhe 1º elemento					
	Imprime valor	2				
	Insere subárvores esq e dir					
[31]	Recolhe 1º elemento					
	Imprime valor	8				
	Insere subárvores esq e dir					
[]	Recolhe 1º elemento					
	Imprime valor	31				
	Insere subárvores esq e dir					